

驾驭巨型代码库

从快速破局未知系统，到 AI 时代的架构重塑指南



面对百万行未知代码，真正的挑战不是阅读量而是策略



新手的迷茫

试图从头到尾阅读源码，最终迷失在无尽的文件跳转与隐式依赖中。



经验丰富的老手

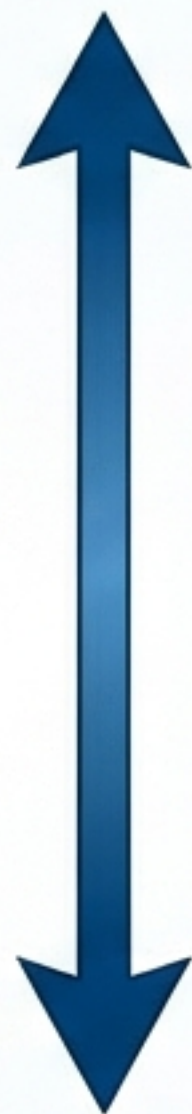
- 寻找立足点：从具体的输入和输出开始构建认知边界。
- 敢于试错：在公开场合犯错是一项被严重低估的超能力。

建立系统级的深刻认知需要双核并行运作



自上而下 (Top-Down)
寻找系统边界与全貌

- 明确业务目标，梳理高层架构，分析 API 契约与集成测试。



自下而上 (Bottom-Up)
建立微观立足点与认知

- 定位核心函数，追踪特定 Bug，分析底层日志与单一执行路径。

不断在两者之间切换，直到微观细节与宏观架构在脑海中闭合。

从微观切入：将调试器作为探索未知领域的探测器

条件断点 (Conditional Breakpoints)

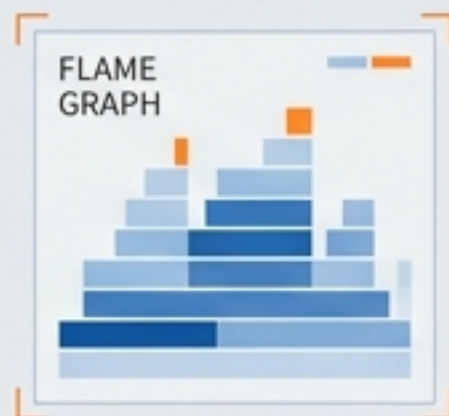
拦截特定状态，沿调用栈逆向追溯逻辑源头。

交互式探索

视调试器为编译型语言的交互式解释器 (REPL)。

性能剖析 (Profiling)

使用火焰图分析执行路径，直观发现核心代码区。



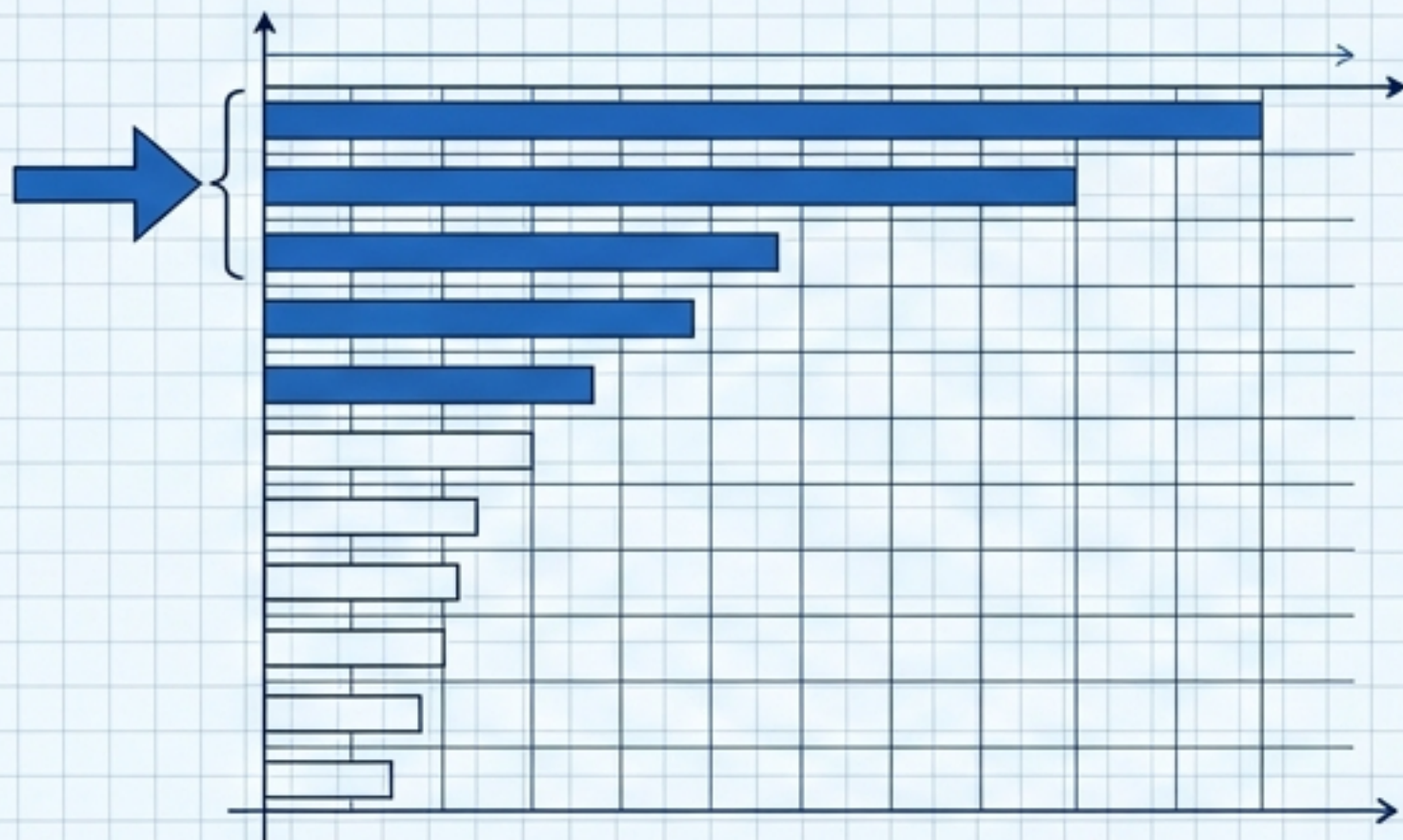
单一请求追踪

从 `main()` 函数或具体 API 开始，记录完整请求流。



从宏观俯瞰：利用版本控制与可视化工具绘制代码藏宝图

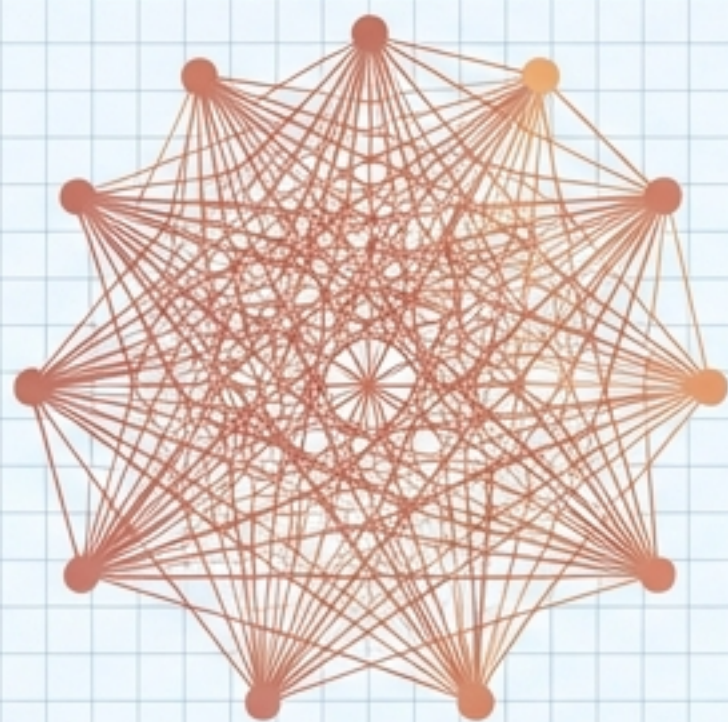
挖掘 Git 历史 (80/20 法则)



执行 `git log --name-only | sort | uniq -c | sort -rg | head -10`

找出修改最频繁的核心文件与技术债务重灾区。

透视依赖图谱

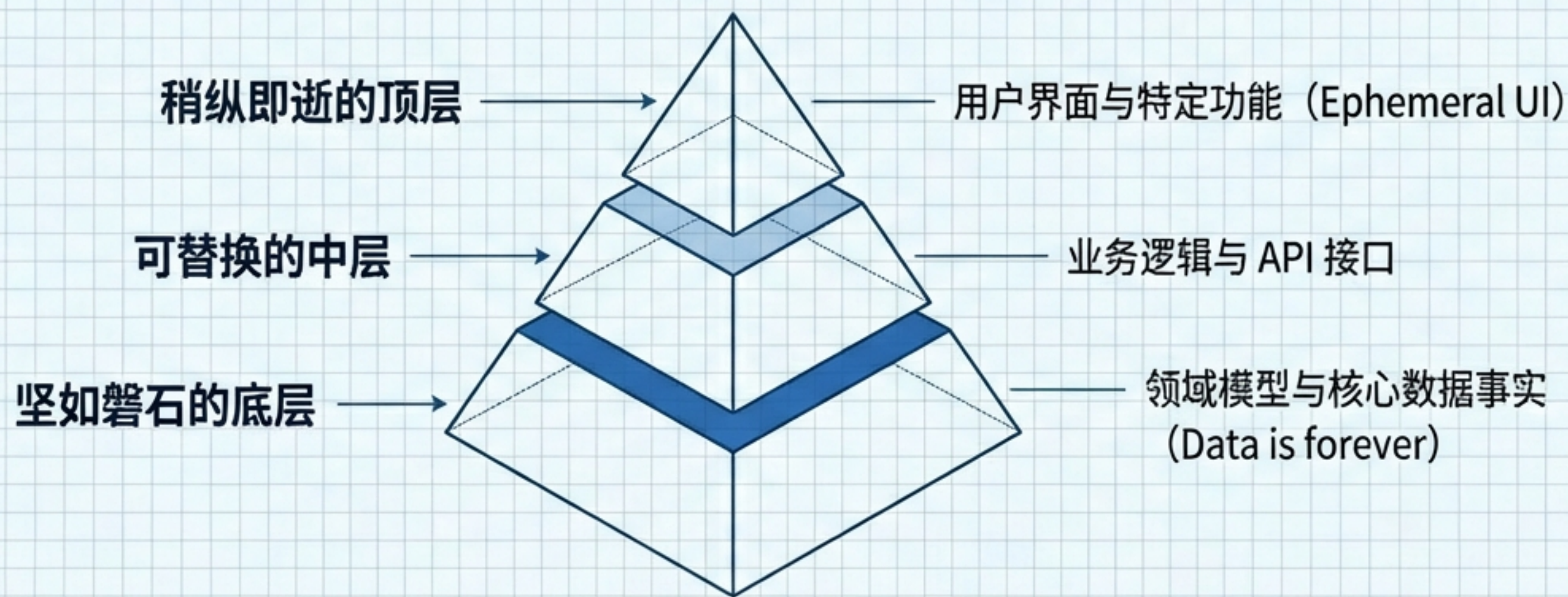


清晰的模块边界

使用 Emerge 等可视化工具分析架构，识别“大泥球”与清晰的模块边界。

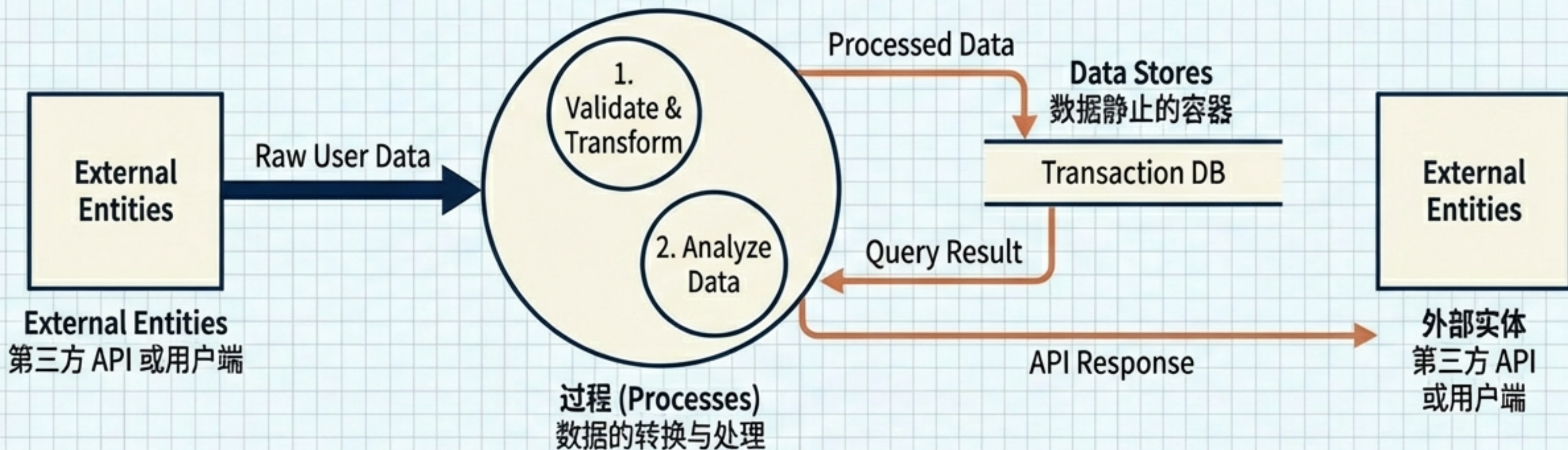
卓越的系统通常源于自上而下的设计与自下而上的编码

“Top-down design, bottom-up coding.”



静态代码无法诉说完整故事，追踪数据的动态流向才是关键

复杂的系统掩盖了数据的真实移动轨迹。使用数据流图（DFD）还原系统真相。



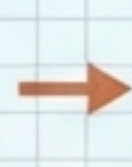
过程 (Processes) -
数据的转换与处理



数据存储 (Data Stores) -
数据静止的容器

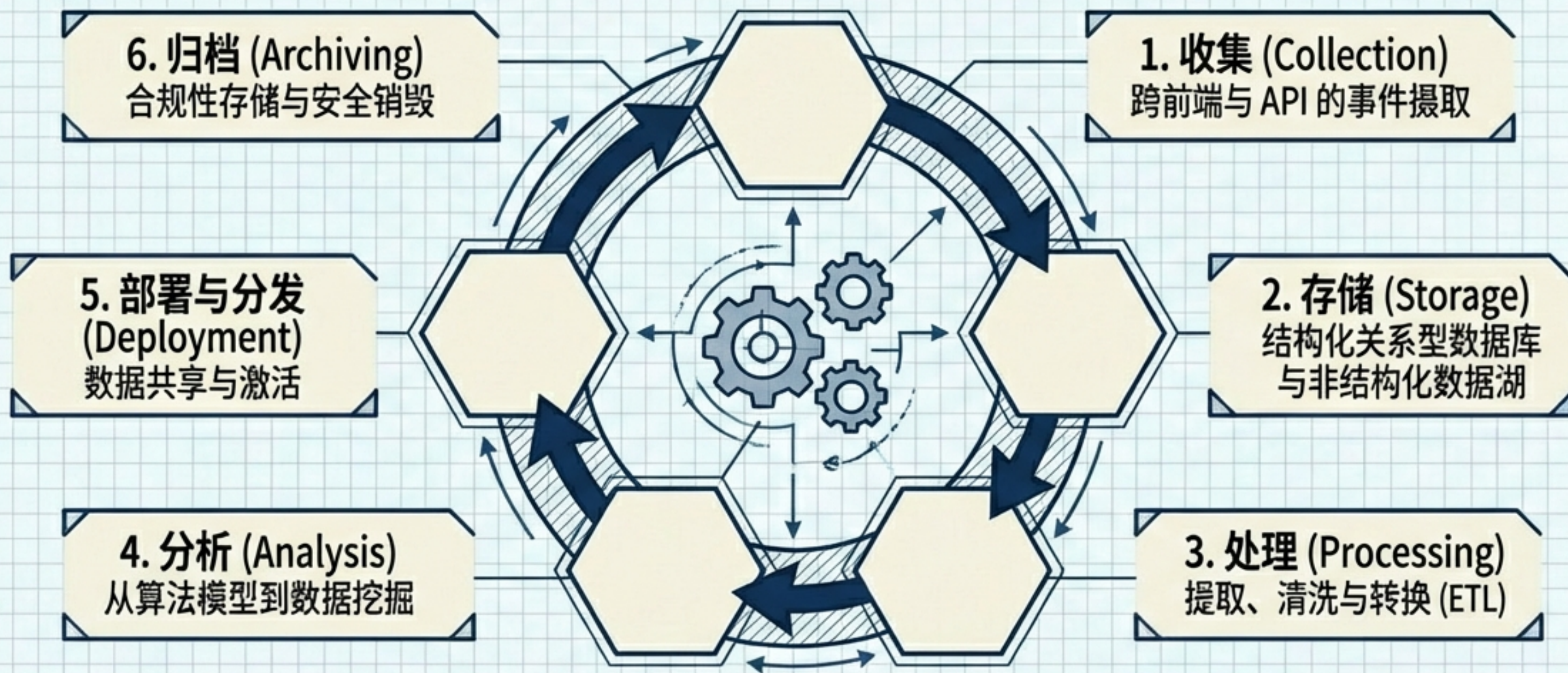


外部实体 (External Entities) -
第三方 API 或用户端



数据流 (Data Flows) -
带有明确标签的定向流向

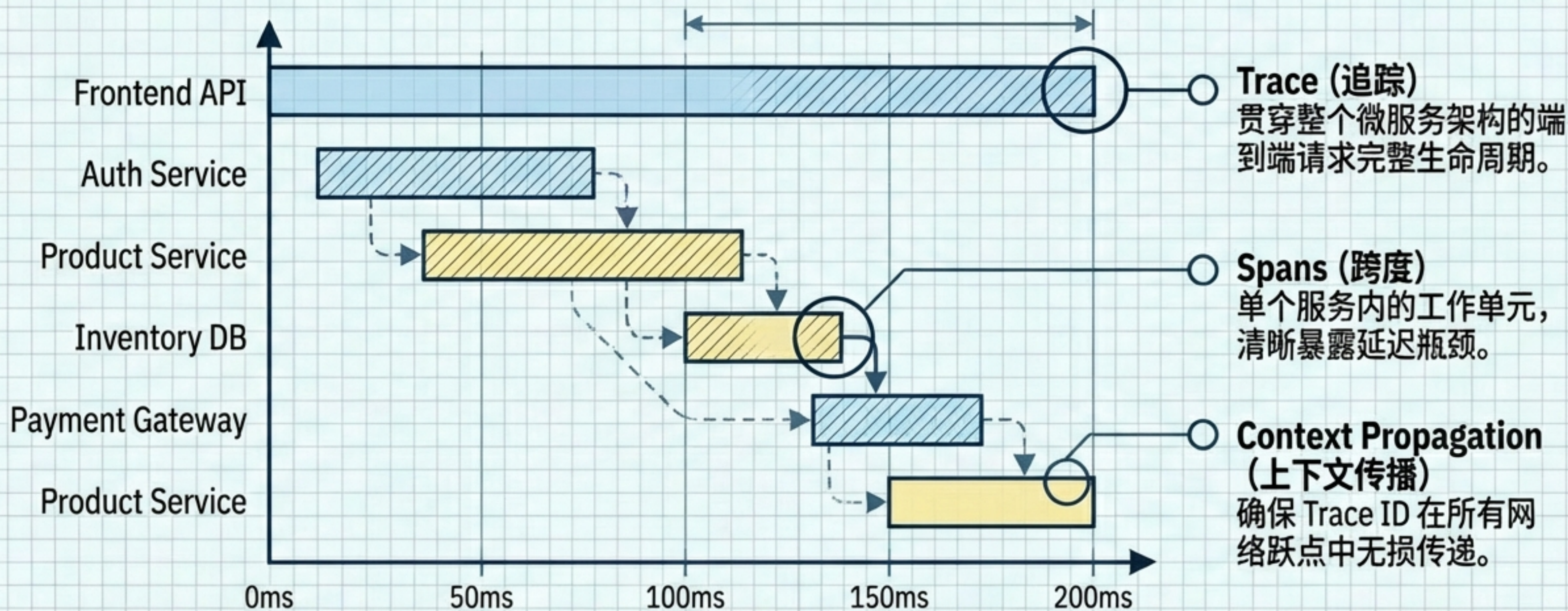
代码库本质上是服务于数据生命周期的精密容器



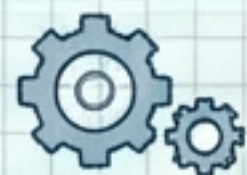
清晰的生命周期管理是打破数据孤岛、消除重复冗余记录的前提。

现代架构要求我们拥有跨越微服务边界的透视能力

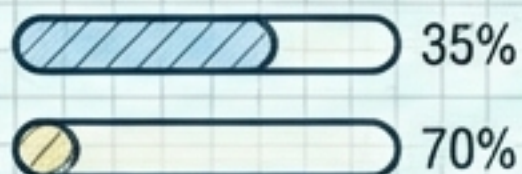
单个前端点击可能触发数十个微服务的连锁反应，传统的单机日志已无法追踪全貌。



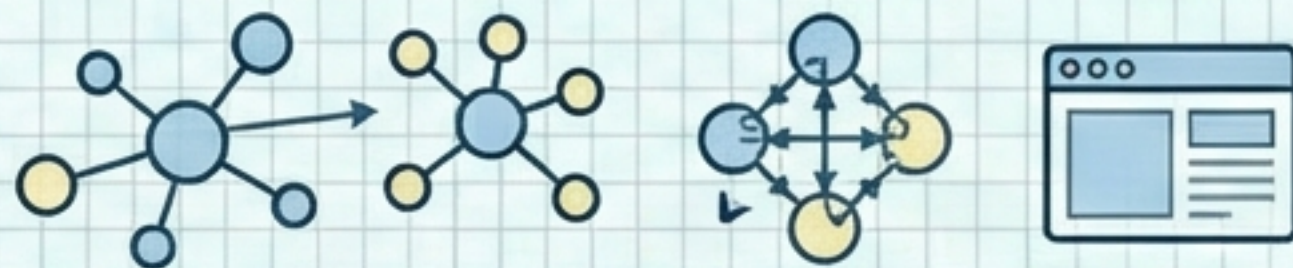
现代开发者的全栈导航与协作工具箱



全局洞察: Sourcegraph Code Insights
将代码库转化为可查询的数据库, 实时追踪速迁移进度、漏洞与代码异味。



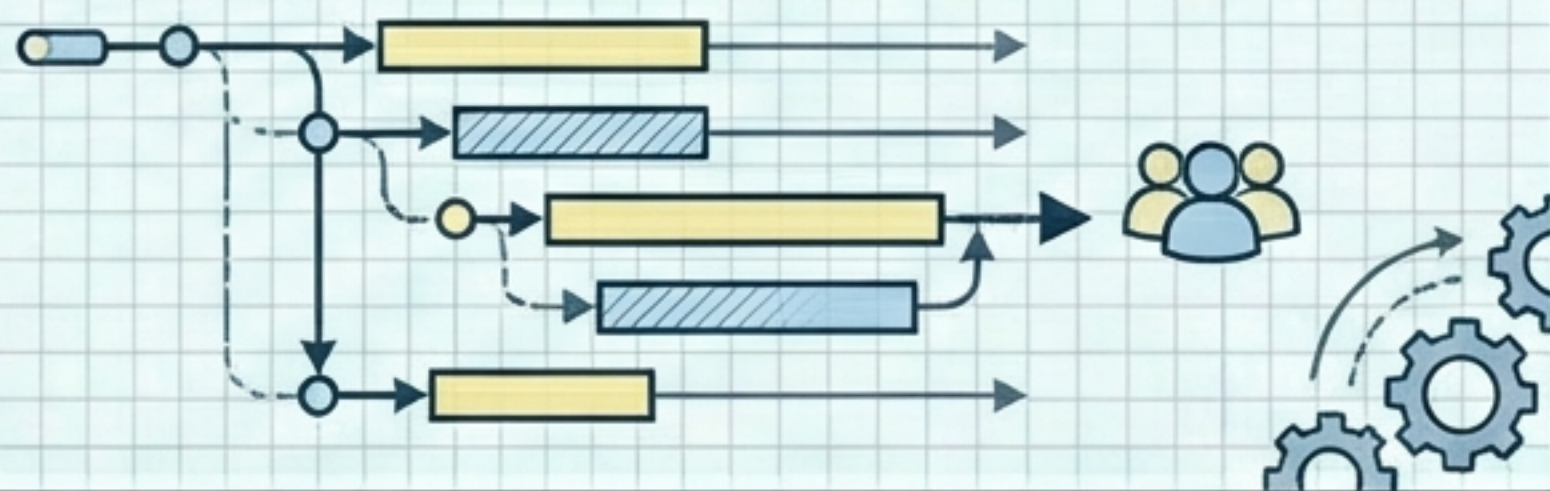
架构可视化: Emerge
在浏览器中交互式探索代码库依赖, 基于 Louvain 模块化算法进行图结构聚类。



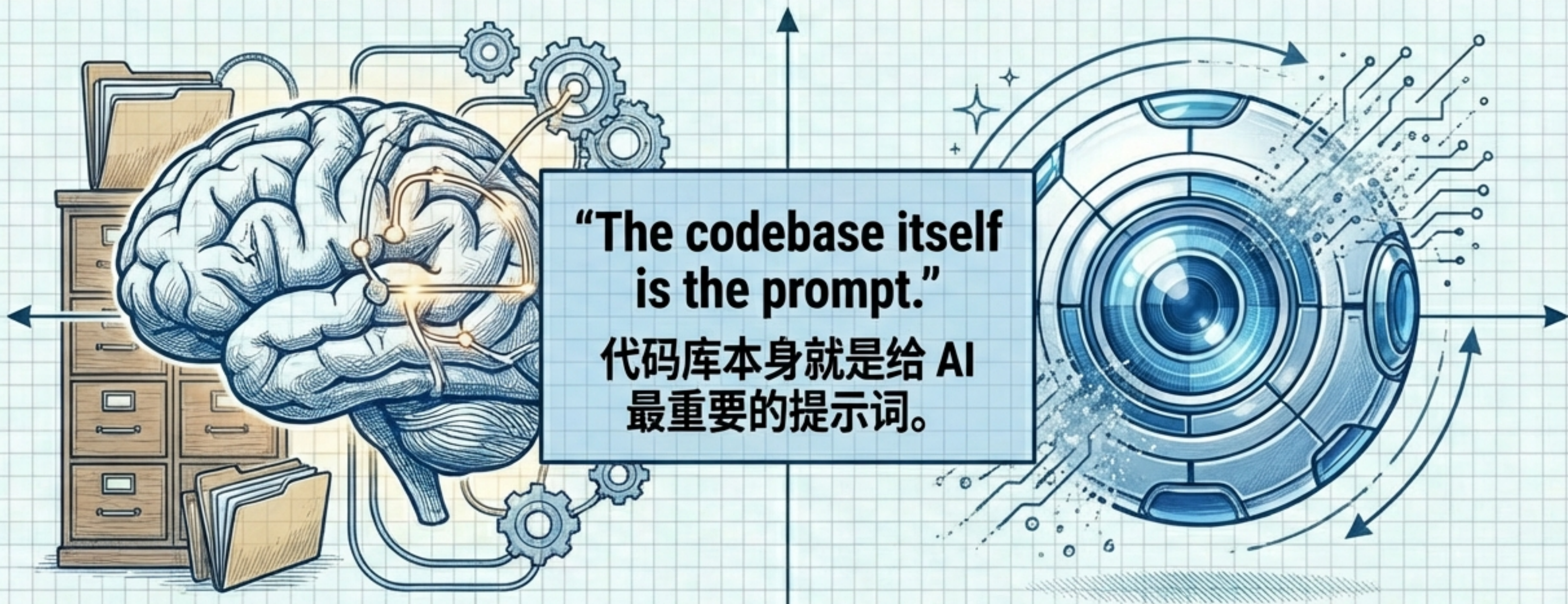
IDE 武器库: IntelliJ vs. VS Code
重量级 JVM 生态的深度重构利器 vs 轻量级、高度可定制的多语言扩展平台。



workflow 加速: Graphite
为高频迭代团队打造, 通过堆叠式 PR (Stacked PRs) 加速 Git 工作流与代码审查。



范式转移：你的下一个团队成员是每天都会失忆的 AI



**“The codebase itself
is the prompt.”**
代码库本身就是给 AI
最重要的提示词。

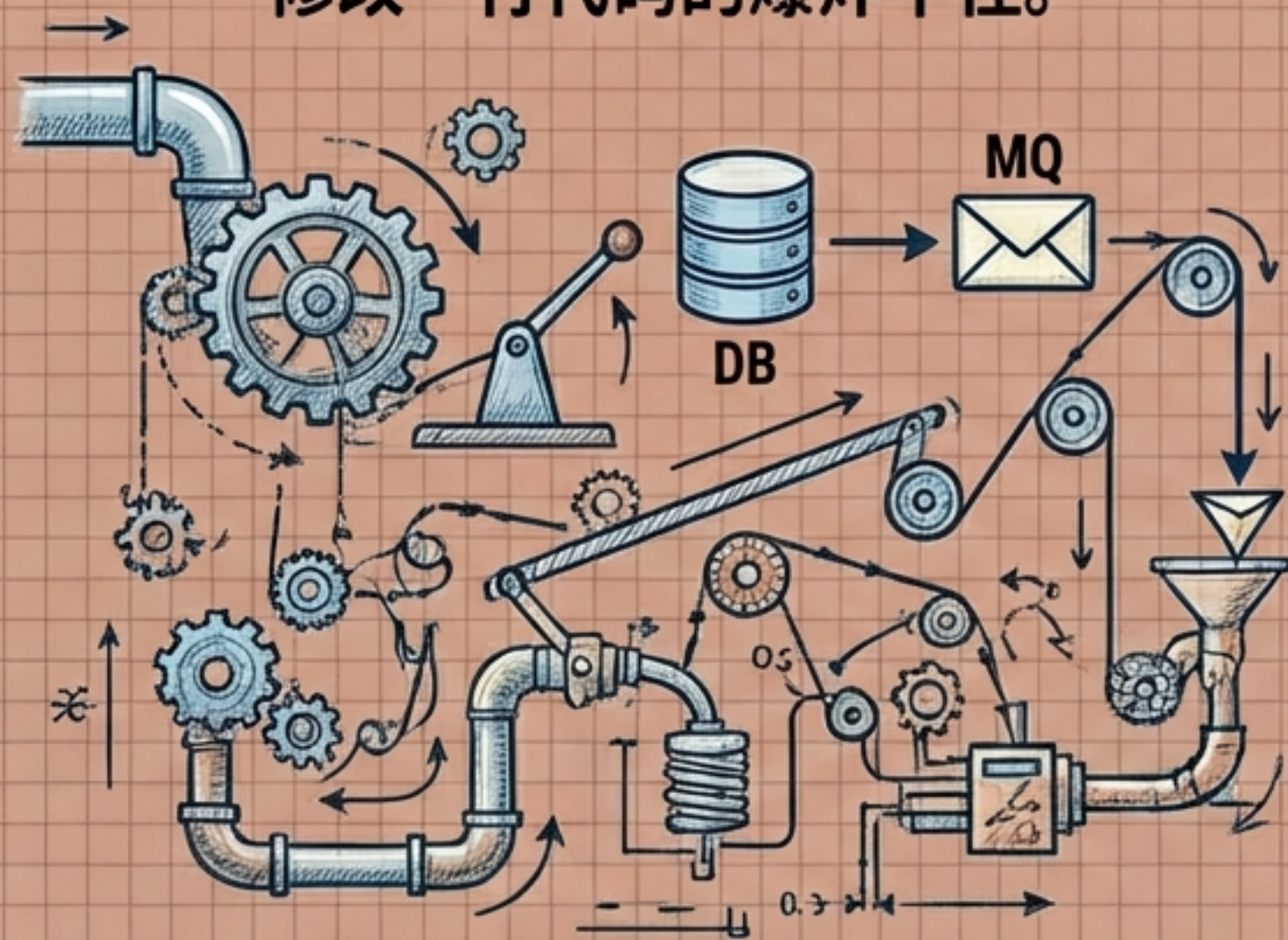
**人类团队：依靠部落记忆 (Tribal Knowledge)
填补隐晦的依赖关系。**

**AI 智能体：每次会话都在从零开始，
没有任何历史上下文。**

面向 AI 的架构首要原则：构建接收器，而非管道

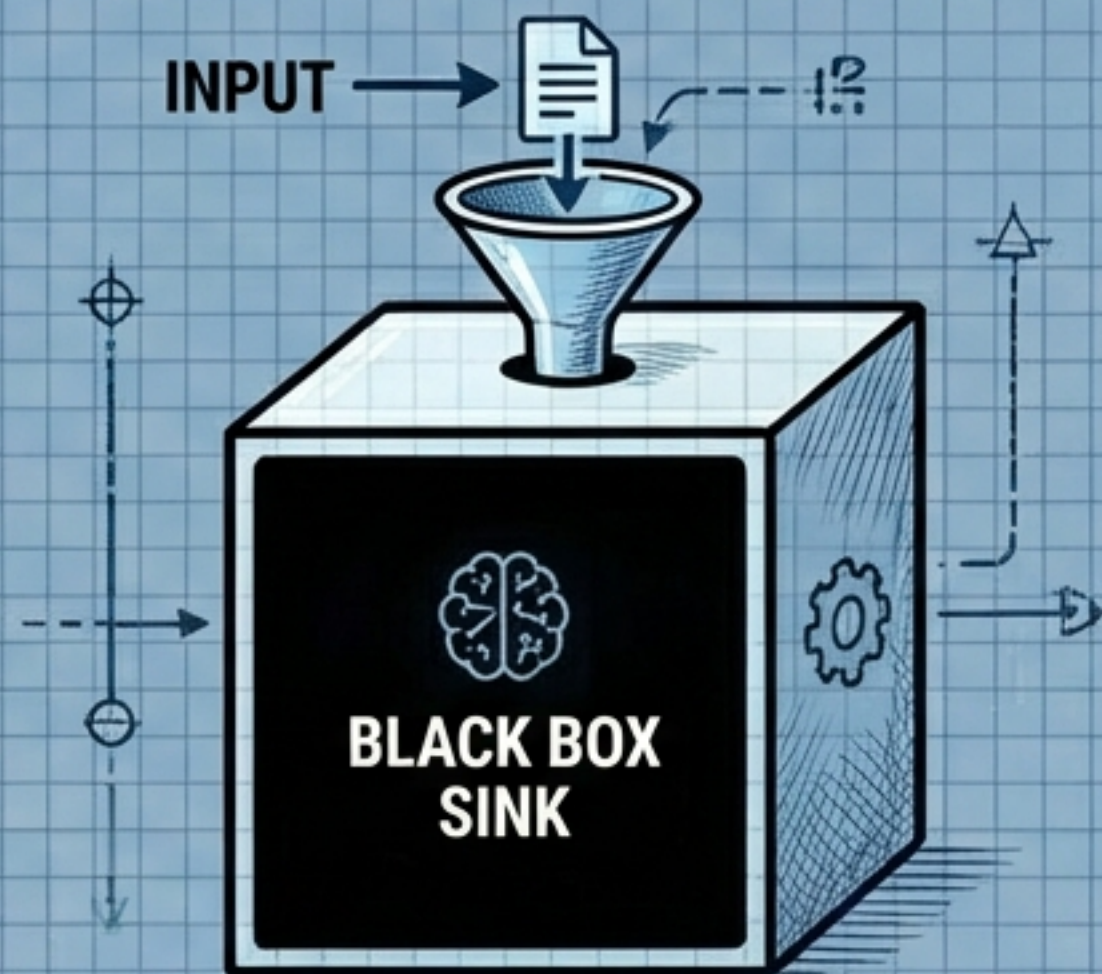
管道式 (Pipes) - 反模式

隐式耦合，副作用无穷。AI 无法预测
修改一行代码的爆炸半径。

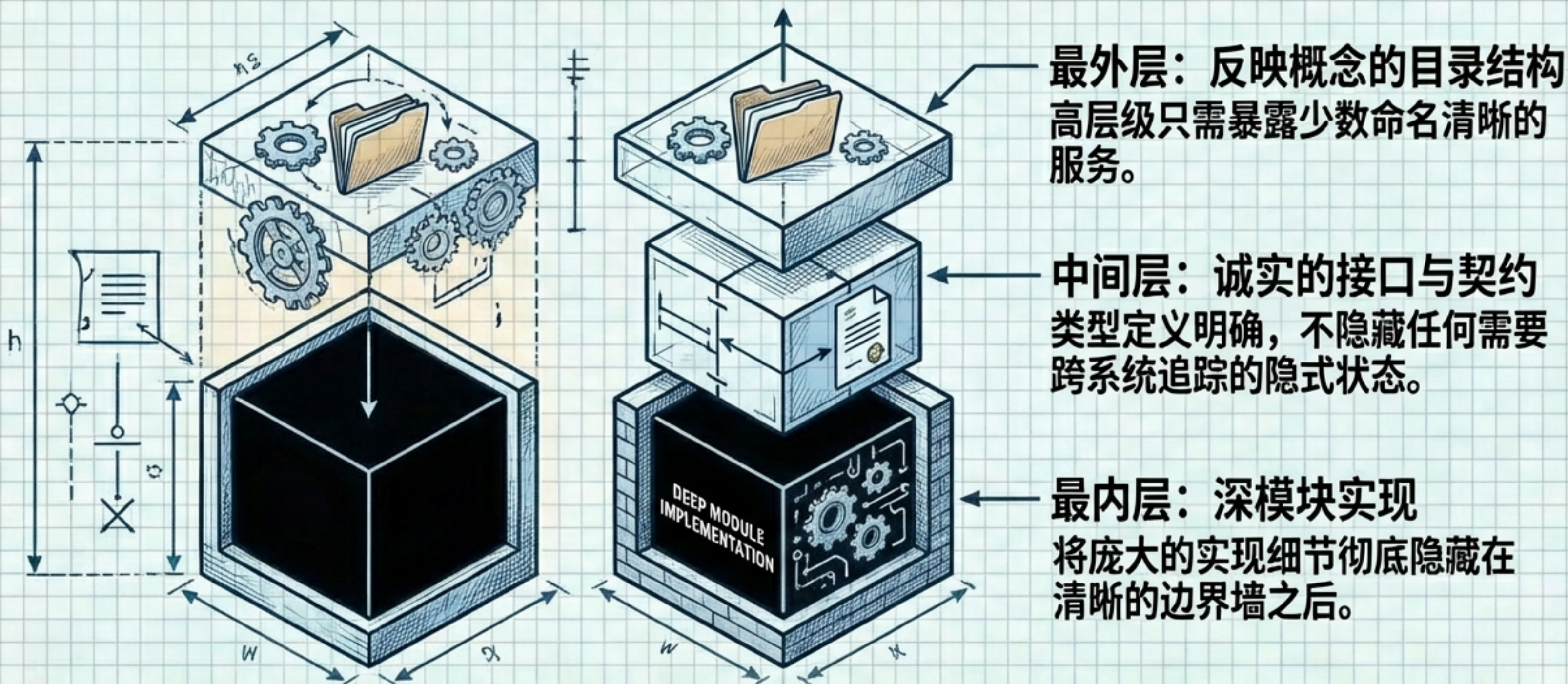


接收器式 (Sinks) - 最佳实践

深度封装。AI 可以将其作为独立黑盒
进行推理、修改和测试，安全可控。



渐进式呈现复杂性是确保 AI 智能体稳定运行的基石



让 AI 能够由外向内构建工作心智模型，而无需强制阅读每一行底层源码。

驾驭代码巨兽的终极生存指南



从任务切入，拒绝盲读

利用调试器追踪特定的请求流或 Bug，将其作为交互探索的平台。

绘制宏观蓝图

使用可视化工具或数据流图透视高层架构与数据真实走向。

消除隐晦的魔法

摒弃部落记忆，把依赖关系显式化，静态代码分析工具和 AI 看不懂潜规则。

面向 AI 重塑边界

打造高内聚的接收器架构，阻断无序的副作用涟漪。

优秀的代码库不仅是机器的指令， 更是给未来自己的完美提示词

当你为每天都会失忆的 AI 打造出边界清晰、无需背景知识也能理解的系统时……
……你会发现，这也正是人类开发者梦寐以求的终极架构。